

GPUMODE Lecture Study Notes: Lecture 100: InferenceX Continuous OSS Inference Benchmarking

Core Problem the Lecture Addresses

The lecture explains **InferenceX**, a continuous open-source inference benchmarking platform built by SemiAnalysis. The central problem is that modern large language model inference cannot be understood by a single number such as maximum throughput. Real deployments operate along a **latency-throughput Pareto frontier**, where providers choose different operating points depending on user experience, serving cost, power, total cost of ownership, model architecture, batch size, tensor parallelism, expert parallelism, precision, and memory capacity.

A static benchmark script hidden on a local machine has limited ecosystem impact. InferenceX is designed as a **living benchmark**: it runs continuously, incorporates new inference engine releases, tracks frontier movement over time, and exposes results publicly so that vendors, cloud providers, inference engine teams, and users can compare realistic serving trade-offs.

The lecture focuses on five interconnected themes:

- Why inference benchmarking must measure the full latency-throughput frontier rather than only peak throughput.
- How continuous benchmarking infrastructure can accelerate software and hardware competition.
- Why model architecture trends such as sparse mixture-of-experts, fewer KV heads, multi-head latent attention, sparse attention, and multi-token prediction reshape serving systems.
- How modern serving techniques such as disaggregated prefill/decode, wide expert parallelism, and speculative decoding change cost and throughput.
- How benchmark data can be translated into real-world economic reasoning about cost per million tokens and inference provider margins.

Required Background

Inference Serving Stages

Autoregressive transformer inference has two main stages.

Prefill

During prefill, the model processes the entire input prompt. If the input sequence length is L_{in} , the model consumes all prompt tokens and builds a key-value cache for future decoding.

Prefill tends to be **compute-bound**, especially for long prompts, because the model performs large matrix multiplications over many tokens. The main user-facing metric affected by prefill is:

$$\text{TTFT} = \text{time to first token.}$$

A lower TTFT means the user sees the first generated token sooner.

Decode

During decode, the model generates output tokens one at a time. Each step reads model weights and the KV cache and produces the next token.

Decode tends to be **memory-bound**, especially at low batch sizes, because each generated token requires moving a large amount of model state and KV-cache data relative to the amount of arithmetic performed.

The main user-facing metric affected by decode is:

$$\text{TPOT} = \text{time per output token.}$$

Equivalently, one can measure per-user output speed:

$$\text{TPS}_{\text{user}} = \frac{1}{\text{TPOT}}.$$

Input and Output Sequence Lengths

InferenceX uses workload scenarios defined by input sequence length and output sequence length:

$$\text{ISL} = \text{input sequence length,} \quad \text{OSL} = \text{output sequence length.}$$

The lecture describes three main scenarios:

- 1K/1K: chat-like workload.

- 1K/8K: long decode workload.
- 8K/1K: long prefill workload.

These scenarios do not cover every real production case, but they provide useful baseline slices of the serving space.

Throughput–Latency Pareto Frontier

A **Pareto frontier** shows the best attainable trade-offs between two competing objectives. In inference serving, one objective is usually high throughput, and the other is low latency.

Let:

T = aggregate output throughput in tokens per second,

ℓ = latency or inverse interactivity.

A configuration is Pareto-optimal if there is no other configuration with both higher throughput and lower latency. InferenceX plots many serving configurations and reports the best frontier.

The benchmark is not asking, “What is the maximum throughput?”
It is asking, “Across all useful latency targets, what is the best throughput and cost that this hardware and software stack can deliver?”

Tensor Parallelism

Tensor parallelism partitions matrix operations across multiple GPUs. If the tensor parallel degree is p_{TP} , each GPU holds and computes part of a layer.

Tensor parallelism is useful for reducing latency because more GPUs collaborate on the same request:

$$\text{per-request compute per GPU} \approx \frac{\text{total layer compute}}{p_{TP}}.$$

However, tensor parallelism introduces communication after partitioned operations. It is often better for high-interactivity, low-batch-size serving points.

Expert Parallelism

For mixture-of-experts models, only a subset of experts is activated per token. Expert parallelism partitions experts across GPUs. If there are E total experts and k active experts per token, then the expert activation ratio is:

$$r_{\text{expert}} = \frac{k}{E}.$$

Modern sparse models may have very small r_{expert} , such as 5% or lower. Expert parallelism can improve memory capacity and throughput but requires all-to-all communication to route tokens to the GPUs holding their selected experts.

KV Cache

The KV cache stores attention keys and values for previously processed tokens. For a transformer with L layers, H_{kv} KV heads, head dimension d_h , sequence length S , and element size b bytes, an approximate KV-cache size is:

$$\text{KVBytes} = 2 \cdot L \cdot H_{\text{kv}} \cdot d_h \cdot S \cdot b.$$

The factor 2 appears because both keys and values are stored. Reducing H_{kv} , using latent attention, or sparsifying attention directly reduces serving memory pressure.

Main Concepts

InferenceX as a Continuous Benchmark

InferenceX is described as a continuous open-source inference benchmark that runs daily across a large pool of accelerators, including GPUs and future support for TPU and Trainium results. The benchmark is built to be open, reproducible, and continuously updated.

The platform has performed on the order of 120,000 sweeps, corresponding to multiple GPU-years of benchmarking. Each sweep tests one or more configurations of model, hardware, precision, inference engine, batch size, parallelism, and serving scenario.

The benchmark is intended to push the ecosystem forward:

- Vendors can submit improved recipes.
- Inference engine teams can observe regressions and improvements.
- Users can compare hardware and software stacks across cost, latency, and throughput.
- The public frontier shows whether new releases truly improve serving performance.

Why Maximum Throughput Alone Is Insufficient

Maximum throughput is usually measured at high batch size and high latency. That point may be useful for offline batch generation, but it does not describe interactive serving.

The lecture uses a bus-versus-F1-car analogy:

- A bus has high aggregate throughput because many passengers share the same vehicle, but each passenger experiences slower travel.
- An F1 car has low sharing but high interactivity and low latency.

The analogous serving trade-off is:

large batch size $\Rightarrow$ high utilization and low cost per token,
 small batch size $\Rightarrow$ low latency and high interactivity.

A complete benchmark must therefore show all useful operating points.

Benchmark Orchestration

The benchmark infrastructure is intentionally simple. Jobs are orchestrated through GitHub Actions, and many compute resources are donated clusters. A typical execution flow is:

1. A benchmark sweep is queued.
2. GitHub Actions dispatches jobs to runner listeners.
3. Runner listeners live on login nodes of donated clusters.
4. Jobs are scheduled through cluster managers such as Slurm.
5. Each serving scenario starts the relevant inference server.
6. The benchmark client sends requests.
7. Results are uploaded as artifacts.
8. Artifacts are aggregated and displayed on the InferenceX frontend.

The design goal is reproducibility rather than unnecessary benchmark complexity.

Benchmark Client and Random Data

The current benchmark client uses random data. This is useful as a baseline because it avoids private production traces and allows stable testing across providers. However, random data has important limitations:

- It does not capture prefix caching benefits.
- It may distort multi-token prediction acceptance rates.
- It does not fully model multi-turn chat or agentic workflows.
- It cannot capture provider-specific real production traffic distributions.

To reduce artificial regularity, InferenceX introduces variation in ISL and OSL rather than using a constant GEMM shape throughout the benchmark. Constant shapes can overstate performance because kernels and CUDA graph captures see less variation.

Future Direction: Realistic Traces

The team discussed moving toward more realistic production-style traces. Real production traces are hard to obtain because they are valuable, private, and sensitive. Public datasets such as ShareGPT are older and may no longer represent modern usage.

Possible future directions include:

- synthetic trace generation,
- multi-turn datasets,
- agentic coding workloads,
- more models,
- more accelerator backends,
- better prefix-caching and KV-cache-routing coverage.

The goal is not merely to hill-climb a synthetic benchmark but to make optimizations that improve real workloads.

Hardware-Level Explanation

Single Node, Scale-Out, and Rack-Scale Serving

The lecture distinguishes three serving deployment categories.

Single Node

A single node, such as an 8-GPU HGX system, uses high-bandwidth intra-node interconnects such as NVLink. This setting is common for models that fit comfortably within one node.

Scale-Out

Scale-out serving uses multiple nodes connected over a network such as InfiniBand. For example, one might use 9 nodes with 8 GPUs each. This increases aggregate memory capacity and compute but introduces slower cross-node communication.

Rack Scale

Rack-scale systems such as NVL72 connect many GPUs within a single high-bandwidth NVLink domain. The lecture emphasizes that this is critical for wide expert parallelism because expert-parallel all-to-all communication is bandwidth-sensitive.

If a rack-scale system has $G = 72$ GPUs and each GPU contributes memory bandwidth B_{HBM} , then an idealized aggregate bandwidth is:

$$B_{\text{agg}} = G \cdot B_{\text{HBM}}.$$

In practice, realized bandwidth depends on communication topology, collective implementation, routing, synchronization overhead, and workload shape.

Compute-Bound Prefill

During prefill, the model processes many prompt tokens. For a dense matrix multiplication with dimensions $M \times K$ multiplied by $K \times N$, the approximate arithmetic work is:

$$\text{FLOPs} = 2MKN.$$

Large M , corresponding to many prompt tokens, increases arithmetic intensity and can push execution toward compute-bound behavior.

Arithmetic intensity is:

$$I = \frac{\text{FLOPs}}{\text{BytesMoved}}.$$

Under the roofline model, attainable performance is bounded by:

$$P_{\text{attainable}} = \min(P_{\text{peak}}, B_{\text{mem}} \cdot I),$$

where P_{peak} is peak compute throughput and B_{mem} is memory bandwidth.

Memory-Bound Decode

During decode, each request typically contributes only one new token per step. At low batch size, the GPU repeatedly streams weights and KV-cache data but performs relatively little arithmetic per memory byte.

This creates low arithmetic intensity:

$$I_{\text{decode}} \ll I_{\text{prefill}}.$$

As a result, decode often lies in the memory-bound region:

$$P_{\text{decode}} \approx B_{\text{mem}} \cdot I_{\text{decode}}.$$

Increasing batch size improves reuse of loaded weights across multiple tokens:

$$\text{effective bytes per token} \downarrow \quad \text{as} \quad \text{batch size} \uparrow.$$

This is why high-throughput serving usually uses larger batches and lower interactivity.

Why Disaggregation Helps

In a non-disaggregated system, the same GPU pool handles prefill and decode. Since prefill and decode have different bottlenecks, one stage can interfere with the other:

- prioritizing decode improves TPOT but can worsen TTFT,
- prioritizing prefill improves TTFT but can worsen decode smoothness.

Disaggregated serving assigns separate resources:

$$\mathcal{G}_{\text{prefill}} \cap \mathcal{G}_{\text{decode}} = \emptyset.$$

A request first enters the prefill pool, which computes prompt activations and KV cache. Then the KV cache is transferred to the decode pool, which completes generation.

The cost is KV-cache movement:

$$T_{\text{transfer}} \approx \frac{\text{KVBytes}}{B_{\text{link}}} + T_{\text{overhead}}.$$

For disaggregation to help, KV-cache transfer must be overlapped or made small enough that it does not dominate:

$$T_{\text{transfer}} < T_{\text{saved by specialization}}.$$

KV Cache Transfer Libraries

The lecture mentions several components related to KV-cache transfer and multi-node serving:

- built-in transfer backends in engines such as vLLM and SGLang,
- Mooncake, which can use CPU DRAM and SSD resources to build a disaggregated KV-cache pool,
- Dynamo, a framework-agnostic multi-node serving framework that can plug into engines such as vLLM, SGLang, and TensorRT-LLM.

A key systems issue is that moving KV cache is not simply a memcpy. It interacts with request routing, scheduling, cache locality, network topology, and overlap with compute.

Wide Expert Parallelism

Mixture-of-experts models increasingly use many total experts with low activation ratios. For E total experts and k active experts, each token only touches a small fraction:

$$r_{\text{expert}} = \frac{k}{E}.$$

If $E = 256$ and $k = 4$, then:

$$r_{\text{expert}} = \frac{4}{256} = 0.015625 = 1.5625\%.$$

Wide expert parallelism spreads experts across many GPUs. This increases memory capacity and can reduce per-GPU weight loading, but it requires routing tokens to remote experts.

The main communication primitive is all-to-all. If tokens are distributed across GPUs and experts are sharded, then each GPU must send token activations to the GPUs that own the selected experts and receive outputs back.

The effectiveness of wide expert parallelism depends heavily on:

- all-to-all bandwidth,
- batch size,
- token distribution across experts,
- expert load balance,
- overlap between communication and expert computation,
- interconnect topology.

When Tensor Parallelism Beats Expert Parallelism

At low batch size, not enough tokens may be present to activate all experts. If many experts are idle, wide expert parallelism wastes capacity and pays all-to-all overhead.

In high-interactivity low-batch scenarios:

few tokens $\Rightarrow$ few active experts $\Rightarrow$ poor EP utilization.

Tensor parallelism can be better because it allocates multiple GPUs to one request and reduces latency.

At high-throughput operating points, many tokens are in flight:

many tokens $\Rightarrow$ many active experts $\Rightarrow$ better EP utilization.

The lecture notes that wide expert parallelism is especially beneficial in the middle of the curve, where communication can be amortized and additional KV-cache capacity matters.

Visual Concept

Draw a latency–throughput graph. On the right side, label low latency and high interactivity with small batch size and higher tensor parallelism. On the left side, label high throughput with larger batch size and wider expert parallelism. In the middle, show the region where rack-scale NVLink and wide expert parallelism provide the largest benefit.

Mathematical Reasoning

Cost Per Token

A core purpose of InferenceX is translating throughput into cost. Suppose a system has total cost per second:

$$C_{\text{sec}} = \frac{C_{\text{hour}}}{3600}.$$

If the system produces $T_{\text{tok/sec}}$ tokens per second, then the cost per token is:

$$C_{\text{token}} = \frac{C_{\text{sec}}}{T_{\text{tok/sec}}}.$$

Cost per million tokens is:

$$C_{1\text{M}} = 10^6 \cdot \frac{C_{\text{hour}}}{3600 \cdot T_{\text{tok/sec}}}.$$

This can be computed separately for input tokens and output tokens if the benchmark reports prefill and decode throughput or if a TCO model assigns costs to each stage.

Provider Margin Estimate

If a provider charges $P_{1\text{M}}$ dollars per million tokens and the estimated TCO cost is $C_{1\text{M}}$, then gross margin is:

$$\text{Margin} = 1 - \frac{C_{1\text{M}}}{P_{1\text{M}}}.$$

In the lecture example, OpenRouter pricing for a DeepSeek R1 FP8 endpoint is compared with InferenceX-derived cost estimates at approximately 35 tokens per second per user. This allows a rough margin estimate.

This is explicitly a napkin calculation because it does not fully include:

- downtime,

- request routing inefficiency,
- idle capacity,
- engineering cost,
- networking cost,
- power delivery overhead,
- utilization variation,
- prefix caching,
- real traffic burstiness.

Speculative Decoding Speedup

Speculative decoding uses a draft mechanism to propose multiple tokens and a target model to verify them. If each target-model decode pass verifies and accepts an expected a tokens, then the ideal token-generation speedup is approximately:

$$S_{\text{spec}} \approx a.$$

More precisely, if one target pass costs T_{target} , draft cost is T_{draft} , and the expected accepted tokens per cycle is a , then:

$$\text{TPOT}_{\text{spec}} \approx \frac{T_{\text{target}} + T_{\text{draft}}}{a}.$$

The speedup relative to standard decoding is:

$$S_{\text{spec}} = \frac{T_{\text{target}}}{(T_{\text{target}} + T_{\text{draft}})/a} = \frac{aT_{\text{target}}}{T_{\text{target}} + T_{\text{draft}}}.$$

If the draft mechanism is cheap and $T_{\text{draft}} \ll T_{\text{target}}$, then:

$$S_{\text{spec}} \approx a.$$

Multi-Token Prediction

Multi-token prediction integrates draft heads or modules into the model architecture. Instead of using a separate draft model, MTP modules are trained with the main model and propose future tokens.

The lecture notes that MTP provides the largest gains at high-interactivity, low-concurrency points. The reason is that GPUs are underutilized in ordinary low-batch decode. Verifying several tokens in one pass uses otherwise idle compute.

At high batch size, the GPU is already saturated:

high batch size $\Rightarrow$ high utilization $\Rightarrow$ smaller MTP marginal gain.

Sparse Attention Complexity

Dense autoregressive attention over a KV cache of length S requires attending over all previous positions. Per decode step, attention cost scales approximately as:

$$\mathcal{O}(S).$$

DeepSeek Sparse Attention-style methods use an indexer and top- K selector to choose only K positions. If K is fixed, such as $K = 2048$, then decode attention cost becomes:

$$\mathcal{O}(K).$$

For fixed K , with respect to total context length:

$$\mathcal{O}(K) = \mathcal{O}(1).$$

This does not mean attention is free; it means attention cost no longer grows linearly with the full context length once K is fixed.

KV-Head Reduction

For traditional multi-head attention, KV-cache size scales with the number of KV heads:

$$\text{KVBytes} \propto H_{\text{kv}}.$$

For grouped-query attention, multi-query attention, or multi-head latent attention, H_{kv} is reduced or compressed. This reduces:

- memory capacity pressure,
- KV-cache transfer size,
- decode memory bandwidth,
- long-context serving cost.

Code Walkthrough

Representative Benchmark Sweep Orchestration

The lecture describes a simple benchmark orchestration flow. The following pseudocode reconstructs the core logic.

```

def run_sweep(sweep_config):
    server = launch_inference_server(
        engine=sweep_config.engine,
        model=sweep_config.model,
        precision=sweep_config.precision,
        tensor_parallel=sweep_config.tensor_parallel,
        expert_parallel=sweep_config.expert_parallel,
        batch_config=sweep_config.batch_config,
    )

    wait_until_ready(server)

    results = []
    for scenario in sweep_config.scenarios:
        requests = make_requests(
            input_length_distribution=scenario.input_lengths,
            output_length_distribution=scenario.output_lengths,
            random_tokens=True,
            apply_chat_template=scenario.apply_chat_template,
        )

        metrics = run_benchmark_client(
            server_url=server.url,
            requests=requests,
            concurrency=scenario.concurrency,
            duration_seconds=scenario.duration_seconds,
        )

        results.append(metrics)

    upload_artifacts(results)
    stop_server(server)

```

The important hardware behavior is hidden inside `launch_inference_server`. That call determines whether the server uses tensor parallelism, expert parallelism, disaggregated serving, CUDA graphs, specialized kernels, MTP, FP8, FP4, or a vendor-specific engine.

The benchmark client itself does not need to be complex. Its job is to generate enough controlled load to expose the performance frontier.

Representative Latency–Throughput Frontier Extraction

A sweep produces many candidate configurations. The Pareto frontier can be extracted by sorting configurations and keeping only non-dominated points.

```

def pareto_frontier(points):
    # Each point has:
    #   latency: lower is better
    #   throughput: higher is better
    points = sorted(points, key=lambda p: p.latency)

    frontier = []
    best_throughput = float("-inf")

    for p in points:
        if p.throughput > best_throughput:
            frontier.append(p)
            best_throughput = p.throughput

    return frontier

```

This logic reflects the benchmark philosophy. A point is only useful if it gives higher throughput than all lower-latency points before it. Otherwise, another configuration dominates it.

Representative Disaggregated Serving Flow

The core disaggregated serving pattern is:

```

def serve_request_disaggregated(request):
    prefill_worker = prefill_pool.pick_worker(request)
    kv_cache, first_token_state = prefill_worker.run_prefill(
        input_tokens=request.input_tokens
    )

    decode_worker = decode_pool.pick_worker(request, kv_cache_metadata=kv_cache.meta)

    transfer_kv_cache(
        source=prefill_worker,
        destination=decode_worker,
        kv_cache=kv_cache,
    )

    output_tokens = decode_worker.run_decode(
        first_token_state=first_token_state,
        kv_cache=kv_cache,
        max_output_tokens=request.max_output_tokens,
    )

    return output_tokens

```

The key systems challenge is the `transfer_kv_cache` call. If KV transfer is not

overlapped with scheduling, prefill, or decode work, then disaggregation may lose its advantage. The ideal implementation pipelines transfer so that decode workers receive cache data just in time.

Representative Speculative Decoding

Speculative decoding can be sketched as follows.

```
def speculative_decode(prompt, target_model, draft_model, max_tokens, draft_len):
    tokens = list(prompt)

    while len(tokens) < len(prompt) + max_tokens:
        draft = draft_model.propose(tokens, num_tokens=draft_len)

        target_logits = target_model.forward(tokens + draft)

        accepted = []
        for i, candidate in enumerate(draft):
            verified_token = sample_from_logits(target_logits, position=len(tokens) + i)
            if candidate == verified_token:
                accepted.append(candidate)
            else:
                accepted.append(verified_token)
                break

        tokens.extend(accepted)

    return tokens
```

In MTP-style decoding, the draft mechanism is integrated into the model through trained MTP heads or modules rather than a completely separate draft model.

Representative Cost Calculation

```
def cost_per_million_tokens(cost_per_hour, tokens_per_second):
    cost_per_second = cost_per_hour / 3600.0
    cost_per_token = cost_per_second / tokens_per_second
    return 1_000_000.0 * cost_per_token

def gross_margin(price_per_million_tokens, cost_per_million_tokens):
    return 1.0 - cost_per_million_tokens / price_per_million_tokens
```

This is the kind of calculation used to translate benchmark throughput into economic estimates. The result is only as good as the assumptions about utilization, hardware price, power, networking, and software configuration.

Performance Intuition

Why Blackwell FP4 Looks So Strong

The lecture highlights large gains for GB200 and GB300 FP4 systems relative to H100 FP8 on heavy prefill workloads such as DeepSeek R1 8K/1K. The gains are not merely from using more GPUs. The results are normalized in ways that expose per-GPU or per-system efficiency improvements.

The major reasons include:

- lower precision with NVFP4,
- rack-scale NVLink bandwidth,
- disaggregated prefill/decode,
- wide expert parallelism,
- improved inference kernels,
- better serving recipes.

FP4 reduces memory bandwidth pressure and weight footprint. For a model weight matrix with N parameters, the approximate storage is:

$$\text{Bytes}_{\text{FP8}} \approx N, \quad \text{Bytes}_{\text{FP4}} \approx \frac{N}{2}.$$

Ignoring metadata and scaling factors, FP4 halves the weight bandwidth requirement relative to FP8. Since decode is often memory-bound, this can significantly improve cost per token.

Accuracy Trade-Offs

The lecture mentions that FP4 can have slightly lower accuracy than FP8, but NVFP4 is designed to make the degradation small. The team currently reports some accuracy evaluation, such as GSM8K, and plans to expand to more difficult harnesses.

The important benchmarking principle is:

speedup without accuracy tracking $\neq$ valid model-serving improvement.

A serious inference benchmark must pair performance with accuracy or quality checks.

AMD Versus NVIDIA

The lecture emphasizes that AMD MI355 appears competitive in FP8 scenarios and is improving quickly, especially through upstreamed SGLang work. However,

AMD results can appear more variable because the software stack is less mature and does not yet use the same wide expert-parallel techniques in all cases.

A key point is:

Hardware capability is necessary but not sufficient. In inference serving, software maturity and iteration speed can dominate realized performance.

AMD has strong chips, but to fully close the gap in FP4 and advanced serving regimes, the ecosystem needs:

- tuned expert-parallel kernels,
- robust communication libraries,
- production-grade KV-cache transfer,
- optimized inference engine integration,
- stable upstream support in engines such as SGLang and vLLM.

Why Vendor Engines Can Beat General Engines

The lecture compares engines such as vLLM, SGLang, TensorRT-LLM, Dynamo, and AMD-specific engines. Vendor-specific engines can be much faster because they are highly specialized for:

- kernel selection,
- CUDA graph capture,
- automatic dependent launches,
- hardware-specific communication,
- memory layout,
- precision formats,
- scheduling constraints.

General engines such as vLLM must support many models, devices, versions, and user configurations. That flexibility can create overhead and prevent aggressive specialization.

A useful mental equation is:

realized performance = kernel quality + scheduling quality + memory management quality + communication quality

The lecture suggests that much of the gap may be kernel-driven, but profiling is required to attribute the exact share.

Profiling Challenges

The discussion notes that profiling inference engines can be difficult because matrix shapes and kernel metadata may not propagate cleanly through profiler traces, especially when kernels bypass certain framework dispatch paths.

For performance engineering, missing shape metadata is costly. Without shapes, it is hard to compute:

$$\text{MFU} = \frac{\text{measured FLOPs per second}}{\text{theoretical peak FLOPs per second}}.$$

It is also hard to determine whether a performance gap comes from:

- GEMM kernels,
- attention kernels,
- MoE routing,
- all-to-all communication,
- KV-cache movement,
- CPU scheduling overhead,
- graph capture misses,
- memory fragmentation.

Why MTP Helps Most at Low Concurrency

At low concurrency, the GPU is underutilized. Standard autoregressive decoding may launch many small operations that do not fill the GPU. MTP increases useful work per decode pass by verifying multiple future tokens.

If the expected accepted tokens per decode pass is $a = 2.5$, then an idealized speedup is close to $2.5\times$, provided the draft overhead is small. The lecture reports that cost improvements can approach roughly $3\times$ in favorable regions.

At high concurrency, the GPU is already saturated:

$$\text{utilization} \approx 1.$$

Then MTP has less room to improve throughput because the system is no longer dominated by idle decode latency.

Common Misconceptions

Misconception: Peak Throughput Is the Benchmark

Peak throughput is only one operating point. Interactive serving requires understanding the whole frontier. A model provider may intentionally run at

lower aggregate throughput to deliver faster tokens per user.

Misconception: Bigger Batch Size Is Always Better

Bigger batch size improves utilization and cost per token but worsens latency. The right batch size depends on service-level objectives.

Misconception: Disaggregation Is Free

Disaggregation separates compute-bound prefill from memory-bound decode, but it introduces KV-cache transfer. If transfer is slow or poorly overlapped, disaggregation can lose its advantage.

Misconception: Expert Parallelism Always Wins for MoE

Expert parallelism is powerful at high throughput and with many tokens in flight. At low batch size, too few experts may be active, and all-to-all overhead can dominate. Tensor parallelism may be better for low latency.

Misconception: Random Data Fully Represents Production

Random data gives a useful baseline, but real workloads include prefix reuse, multi-turn conversations, skewed prompt lengths, tool calls, coding traces, and semantic structure that affects MTP acceptance.

Misconception: FP4 Speedups Are Purely Marketing

The lecture argues that FP4 plus rack-scale serving and advanced software techniques can produce real cost and throughput improvements. However, accuracy evaluation remains necessary.

Misconception: Hardware Alone Determines Inference Cost

Software stack quality strongly affects realized performance. Kernels, communication libraries, scheduling, CUDA graphs, and inference-engine integration can create large gaps on the same hardware.

Practical Takeaways

1. Always evaluate inference systems along a latency-throughput frontier, not only at maximum throughput.
2. Separate prefill and decode mentally because they have different bottlenecks and different user-facing metrics.
3. Treat KV cache as a first-class systems object. Its size, transfer cost, routing, and locality shape serving economics.

4. For MoE models, expert activation ratio determines whether expert parallelism is efficient.
5. Rack-scale NVLink domains are especially valuable for wide expert parallelism because all-to-all communication is bandwidth-sensitive.
6. MTP and speculative decoding are most valuable when decode latency leaves GPU compute underutilized.
7. FP4 improves serving cost by reducing weight bandwidth and memory footprint, but it must be paired with accuracy evaluation.
8. Open-source benchmarks are most useful when they continuously track real software releases and expose frontier movement over time.
9. Vendor engines can outperform general engines because they specialize kernels, scheduling, graph capture, and memory management.
10. Benchmark results can be converted into rough business estimates, but utilization, downtime, and operational overhead must be included for serious financial analysis.

Project or Experiment Ideas

Experiment 1: Build a Mini Pareto Frontier

Run a small local benchmark for an open model using vLLM or SGLang. Sweep:

$$\text{batch size} \in \{1, 2, 4, 8, 16, 32\},$$

$$\text{max concurrency} \in \{1, 4, 16, 64\}.$$

Plot:

$$x = \text{latency}, \quad y = \text{tokens/sec}.$$

Extract the Pareto frontier and compare against raw points.

Experiment 2: Prefill Versus Decode Profiling

Create two workloads:

$$8\text{K}/1\text{K} \quad \text{and} \quad 1\text{K}/8\text{K}.$$

Profile both with Nsight Systems or PyTorch profiler. Compare:

- GEMM time,
- attention time,
- memory bandwidth,
- kernel launch count,

- occupancy,
- achieved FLOPs.

Experiment 3: Simulate Disaggregated Serving

Implement a toy scheduler with separate prefill and decode queues. Model prefill as compute-bound and decode as memory-bound. Add a KV-transfer cost:

$$T_{\text{transfer}} = \frac{\text{KVBytes}}{B_{\text{link}}}.$$

Vary B_{link} and determine when disaggregation helps.

Experiment 4: Expert Parallelism Utilization

Simulate MoE routing with $E = 256$ experts and $k = 4$ active experts per token. For batch size B , estimate the number of unique experts activated. Compare utilization as B grows.

A simple approximation for the probability that expert e is not selected by any token is:

$$P(\text{expert } e \text{ unused}) \approx \left(1 - \frac{k}{E}\right)^B.$$

The expected number of active experts is:

$$\mathbb{E}[\text{active experts}] = E \left[1 - \left(1 - \frac{k}{E}\right)^B\right].$$

This explains why low batch size underutilizes wide expert parallelism.

Experiment 5: Speculative Decoding Acceptance Rate

Measure MTP or speculative decoding acceptance on:

- random tokens,
- real chat prompts,
- prompts with chat templates,
- coding prompts,
- math prompts.

Compare expected accepted tokens:

$$a = \mathbb{E}[\text{accepted tokens per verification pass}].$$

Then estimate speedup:

$$S_{\text{spec}} \approx \frac{aT_{\text{target}}}{T_{\text{target}} + T_{\text{draft}}}.$$

Experiment 6: Cost Per Million Tokens

Take a measured throughput number and an assumed GPU cost per hour. Compute:

$$C_{1\text{M}} = 10^6 \cdot \frac{C_{\text{hour}}}{3600 \cdot T_{\text{tok/sec}}}.$$

Compare against public API pricing and estimate gross margin:

$$\text{Margin} = 1 - \frac{C_{1\text{M}}}{P_{1\text{M}}}.$$

Then add sensitivity analysis for utilization:

$$C_{1\text{M},\text{adjusted}} = \frac{C_{1\text{M}}}{u},$$

where u is utilization.

Final Mental Model

InferenceX is best understood as a microscope for the modern inference stack. It does not ask for one magic throughput number. It traces the full latency-throughput-cost frontier as models, kernels, inference engines, precisions, and hardware systems evolve.

The core mental model is:

LLM inference is a resource allocation problem across compute, memory bandwidth, KV-cache capacity, communication bandwidth, and latency targets. Prefill wants compute. Decode wants memory bandwidth. MoE wants expert routing and all-to-all bandwidth. Long context wants KV-cache efficiency. Interactive serving wants low latency. Cheap serving wants high utilization. The best system is the one that chooses the right point on this frontier and continuously updates as software and hardware improve.

For an engineer, the lecture's most important lesson is that inference performance is not a property of a GPU alone. It is an emergent property of:

model architecture+precision+parallelism+interconnect+kernel quality+scheduler design+KV-cache manager

A serious benchmark must therefore be continuous, open, multi-dimensional, and grounded in realistic serving regimes.